

Crime Records of India API Documentation

Backend API reference for frontend integration.

Base URL

Local development: `http://localhost:3000/api`

All protected endpoints require:

```
Authorization: Bearer ACCESS_TOKEN
```

Authentication

Register a user

POST `/auth/register`

```
{
  "username": "testuser",
  "password": "User@123",
  "email": "test@test.com",
  "full_name": "Test User",
  "department": "Analysis"
}
```

User login

POST `/auth/login-user`

```
{
  "username": "testuser",
  "password": "User@123"
}
```

Returns `accessToken`, `refreshToken`, and the user profile.

Admin login

POST `/auth/login-admin`

```
{
  "username": "superadmin",
  "password": "Admin@123"
}
```

Only admins with an active status can log in. The token contains the admin role: `superadmin`, `admin`, or `moderator`.

Other authentication endpoints

- GET `/auth/profile` - return the authenticated token profile.
- POST `/auth/refresh-token` - exchange a valid refresh token for an access token.
- POST `/auth/logout` - revoke a refresh token.

Dataset APIs

Dataset routes expose only dataset tables. Authentication and governance tables are never available through these routes, even when a table name is supplied directly.

- GET /tables - list available dataset tables.
- GET /tables/:table/schema - return column names, data types, and nullability.
- GET /data/:table?page=1&limit=20 - return paginated rows. limit is 1-100.
- GET /data/:table/filter-options/:column - return distinct non-null values.
- POST /data/:table/search - filter, sort, and paginate rows.

Search request example:

```
{
  "filters": {
    "Area_name": "Delhi",
    "Year": 2001
  },
  "page": 1,
  "limit": 20,
  "sortBy": "Year",
  "sortOrder": "DESC"
}
```

All filter and sort columns are validated against the selected table schema. Dataset views and searches are recorded in views_log and search_logs.

Analytics API

GET /analytics/overview

Returns user/admin counts, dataset table count, view/search totals, the most viewed table, and logins during the previous seven days. The response is cached in Redis for five minutes.

Admin Governance APIs

Admin governance routes require a valid admin JWT and the superadmin, admin, or moderator role.

View dataset tables as an admin

Admins use the same authenticated read endpoints as users:

- GET /tables
- GET /tables/:table/schema
- GET /data/:table?page=1&limit=20
- GET /data/:table/filter-options/:column
- POST /data/:table/search

The frontend can render the returned rows and add Edit and Delete actions at the end of each row.

Update one cell in one row

PATCH /admin/table/:table/row

```
{
  "row": {
    "Area_name": "Delhi",
    "Year": 2001,
    "Group_Name": "AT1-Motor Cycles/ Scooters",
    "Sub_Group_Name": "1. Motor Cycles/ Scooters"
  },
  "column": "Auto_Theft_Stolen",
}
```

```
"value": 9999
}
```

The row object identifies the existing row by its current column values. The request changes only the named column and succeeds only when exactly one row matches. If the selector matches multiple rows, it returns 409 and no data or audit record is written. PUT /admin/table/:table/row is an equivalent method alias for clients that do not use PATCH.

Dataset tables currently do not expose primary-key columns, so the frontend should send enough current row values to identify one row. The response includes the changed column, old value, new value, updated row, and auditLogId.

Delete one selected row

DELETE /admin/table/:table/row

```
{
  "row": {
    "Area_name": "Delhi",
    "Year": 2001,
    "Group_Name": "AT1-Motor Cycles/ Scooters",
    "Sub_Group_Name": "1. Motor Cycles/ Scooters"
  }
}
```

Deletion also requires exactly one match. The response includes the deleted row and auditLogId; the complete pre-delete row is stored in the audit log.

There are no exposed bulk update or bulk delete routes. Every mutation is a single-row operation.

Read audit logs

- GET /admin/audit-logs?page=1&limit=20 - list audit records, newest first.
- GET /admin/audit-logs/:id - return one complete audit record.

Supported list filters:

- tableName (or table)
- action=INSERT|UPDATE|DELETE
- adminId
- from and to as ISO dates (YYYY-MM-DD or a full ISO timestamp)

Each audit record contains the admin id and username, table, row identifier, action type, old JSON snapshot, new JSON snapshot, changed columns, IP address, and creation timestamp. Audit writes are part of the same transaction as the dataset mutation, so a failed audit insert rolls back the data change.

Redis cache

Key | Endpoint | TTL

```
all_tables | GET /tables | 30 minutes
schema_{table} | GET /tables/:table/schema | 1 hour
analytics_overview | GET /analytics/overview | 5 minutes
```

HTTP status codes

- 200 - successful request
- 201 - resource created

- 400 - invalid request, column, value, or filter
- 401 - missing or invalid access token
- 403 - authenticated user lacks admin access or admin is inactive
- 404 - dataset table, column, or audit log not found
- 409 - the selected row is not unique
- 429 - rate limit exceeded when the limiter is enabled
- 500 - unexpected server error

Frontend integration notes

- Store access and refresh tokens securely.
- Send the access token in the Authorization header.
- Use schema responses to render dynamic dataset columns.
- Use pagination for every dataset and audit-log list.
- Treat a 409 mutation response as a request to add more current row fields to the row selector; never retry it as a bulk mutation.
- Show the audit-log detail response when an administrator reviews a change.

Local development

```
cd backend
npm install
npm run dev
```

Redis can be started with `docker compose up -d` or the command in `docker_cmd.txt`.